zzedc Database Abstraction Refactoring Plan

Overview

This plan refactors zzedc to support multiple database backends through a unified abstraction layer, eliminating the need for the separate bbl package.

Supported Backends (fully interchangeable):

- SQLite
- DuckDB
- PostgreSQL

All three backends support the complete zzedc feature set. Selection is based on deployment requirements, not feature availability.

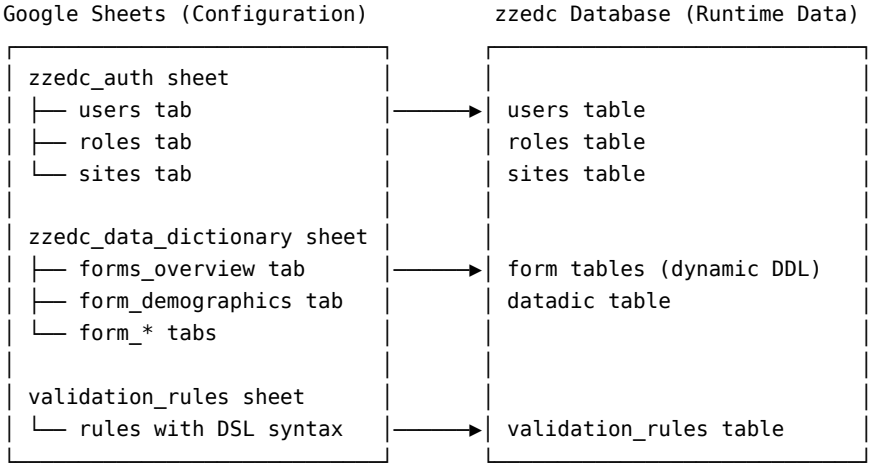
Outcome: bbl becomes redundant; zzedc serves all deployment scenarios with any backend selected via configuration.

Core Architecture

Before discussing database backends, it is important to understand that zzedc has two foundational systems that are central to application operation:

Google Sheets Integration

Google Sheets serves as the **study configuration layer**:



This means:

- **Study setup requires no coding** - Define forms, fields, and validation rules in Google Sheets
- **Changes propagate to database** - Sync from Sheets creates/updates tables
- **Database backend must support dynamic DDL** - All three backends do

Validation System

The validation system uses a **domain-specific language (DSL)** defined in Google Sheets and executed at runtime:

```
Google Sheet Rule                Compiled Validator
─────────────────────────────────────────────────
age >= 18 AND age <= 120    ⟶    R function
```

```
weight > 0                  ⟶   R function
visit_date >= consent_date  ⟶   R function (cross-field)
```

Key components:

- `validation_gsheets_integration.R` - Import rules from Sheets
- `validation_dsl_parser.R` - Parse DSL syntax
- `validation_dsl_codegen.R` - Generate R validators
- `validation_executor.R` - Run validators on form submission

The validation system is **database-agnostic** - it compiles to R code, not SQL. This means all three backends work identically for validation.

Implications for Database Abstraction

The database abstraction must ensure:

1. **DDL generation works for all backends** - `gsheets_dd_builder.R` must generate valid CREATE TABLE statements for SQLite, DuckDB, and PostgreSQL
2. **Validation rules table schema is consistent** - Same structure across backends
3. **Google Sheets sync operations use adapter** - All database writes during study setup go through the abstraction layer

---

Backend Selection Guide

Decision Framework

Choose your backend based on deployment architecture, not user count:

Requirement	SQLite	DuckDB	PostgreSQL
Single server (Shiny + DB same machine)	Yes	Yes	Yes
Separate database server	No	No	Yes
Multiple app servers (load balanced)	No	No	Yes
Managed cloud DB (RDS, Cloud SQL)	No	No	Yes
Zero external dependencies	Yes	Yes	No
Fastest analytical queries	Good	Best	Good
Fastest row-by-row inserts	Best	Good	Good
Maximum ecosystem maturity	Best	Moderate	Best
Native Parquet export	No	Yes	No

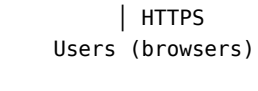
Deployment Scenarios

Desktop / Local Development:

```
Any backend works. SQLite for familiarity, DuckDB for speed.
```

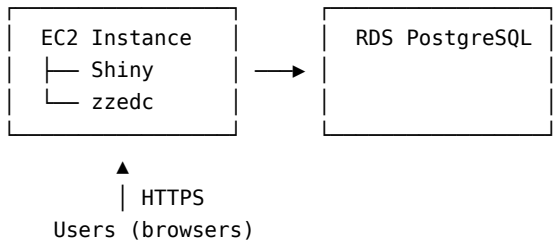
Single EC2/Server with Shiny:

```
┌──────────────────────────┐
│  EC2 Instance            │
│  ├── Shiny Server        │
│  ├── zzedc app           │
│  └── /data/zzedc.db      │  <-- SQLite or DuckDB
└──────────────────────────┘
        ▲
```



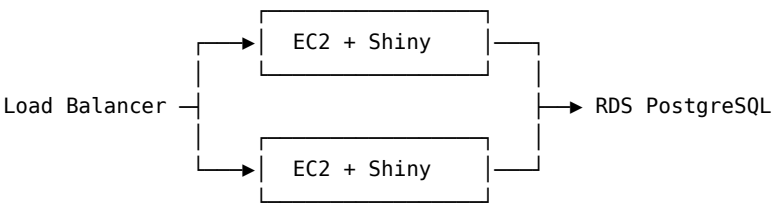
Recommended: DuckDB (fast reports) or SQLite (proven)
Cost: ~$30/month
Concurrent users: 50-100+ easily handled

AWS with Managed Database:



Recommended: PostgreSQL
Cost: ~$45-80/month
Benefits: Managed backups, automatic failover, monitoring

High Availability / Load Balanced:



Required: PostgreSQL (only option for multiple app servers)

Concurrency Clarification

A common misconception is that SQLite/DuckDB cannot handle multiple users. In reality:

Scenario	SQLite/DuckDB	PostgreSQL
50 users via Shiny Server	Works fine	Works fine
50 users direct DB access	Problematic	Works fine
Network file share DB	Risky	Works fine

The key insight: **Shiny Server is a single process accessing the database.** User count matters less than whether multiple processes/servers need simultaneous database access.

Recommendation Summary

Situation	Recommended Backend
Getting started / evaluation	SQLite
Production, single server	DuckDB
Heavy reporting workload	DuckDB
Need Parquet export for statisticians	DuckDB
Multiple app servers required	PostgreSQL

Situation	Recommended Backend
Organization requires “enterprise” DB	PostgreSQL
AWS RDS / managed database	PostgreSQL
Existing PostgreSQL infrastructure	PostgreSQL

Part 1: Current State Analysis

1.1 zzedc Database Usage

Current database operations in zzedc use RSQLite directly:

```
# Typical pattern in zzedc
conn <- RSQLite::dbConnect(RSQLite::SQLite(), db_path)
DBI::dbGetQuery(conn, "SELECT * FROM users")
DBI::dbExecute(conn, "INSERT INTO ...")
DBI::dbDisconnect(conn)
```

1.2 SQLite-Specific Features in Use

Feature	zzedc Usage	PostgreSQL Equivalent
sqlcipher	Encrypted database file	TLS + pgcrypto
AUTOINCREMENT	Primary keys	SERIAL
datetime(‘now’)	Timestamps	CURRENT_TIMESTAMP
		operator
GLOB	Pattern matching	LIKE or ~
JSON functions	json_extract()	-» operator

1.3 Files Requiring Modification

Database operations are concentrated in these files:

File	DB Operations	Complexity
R/db_connection.R	Connection management	High
R/db_migration.R	Schema migrations	High
R/audit_logging.R	Audit trail	Medium
R/electronic_signatures.R	Signatures	Low
R/validation_rules.R	Rule storage	Medium
R/consent.R	Consent records	Low
R/dsar.R	DSAR requests	Low
R/erasure.R	Erasure handling	Low
app/gsheets_dd_builder.R	DDL generation	High
All other R/*.R files	Query execution	Low

Part 2: Database Abstraction Architecture

2.1 Design Principles

1. **DBI as foundation** - All backends use DBI interface

2. **Adapter pattern** - Backend-specific logic in adapter classes
3. **Configuration-driven** - Backend selected via config file
4. **Graceful fallback** - Missing driver = clear error message

2.2 Package Structure

```
zzedc/
├── R/
│   ├── db_adapter.R          # Abstract adapter interface
│   ├── db_adapter_sqlite.R   # SQLite implementation
│   ├── db_adapter_postgres.R # PostgreSQL implementation
│   ├── db_adapter_duckdb.R   # DuckDB implementation
│   ├── db_connection.R       # Refactored to use adapters
│   ├── db_dialect.R          # SQL dialect differences
│   └── ...
```

2.3 Adapter Interface

```
# R/db_adapter.R

#' Create database adapter based on configuration
#' @param config Configuration list with db_backend, db_path, etc.
#' @return Database adapter object
create_db_adapter <- function(config) {


  backend <- config$db_backend %||% "sqlite"

  switch(backend,
    sqlite = SQLiteAdapter$new(config),
    postgresql = PostgreSQLAdapter$new(config),
    postgres = PostgreSQLAdapter$new(config),
    duckdb = DuckDBAdapter$new(config),
    stop("Unsupported database backend: ", backend)
  )
}

#' Abstract database adapter interface
#' @description Base class defining required methods for all adapters
DatabaseAdapter <- R6::R6Class("DatabaseAdapter",
  public = list(
    config = NULL,

    initialize = function(config) {
      self$config <- config
    },

    # Connection management
    connect = function() stop("Subclass must implement connect()
"),
    disconnect = function(conn) DBI
::dbDisconnect(conn),
    pool = function() stop("Subclass must implement pool()"),
    pool_close = function(pool) pool::poolClose(pool),
```

```r
    # Query execution (common via DBI)
    query = function(conn, sql, params = NULL) {
      if (is.null(params)) {
        DBI::dbGetQuery(conn, sql)
      } else {
        DBI::dbGetQuery(conn, sql, params = params)
      }
    },

    execute = function(conn, sql, params = NULL) {
      if (is.null(params)) {
        DBI::dbExecute(conn, sql)
      } else {
        DBI::dbExecute(conn, sql, params = params)
      }
    },

    # Transaction support
    begin = function(conn) DBI::dbBegin(conn),
    commit = function(conn) DBI::dbCommit(conn),
    rollback = function(conn) DBI::dbRollback(conn),

    transaction = function(conn, code) {
      self$begin(conn)
      tryCatch({
        result <- force(code)
        self$commit(conn)
        result
      }, error = function(e) {
        self$rollback(conn)
        stop(e)
      })
    },

    # Schema operations (backend-specific)
    create_table_sql = function(name, columns) {
      stop("Subclass must implement create_table_sql()")
    },

    # SQL dialect helpers
    dialect = function() stop("Subclass must implement dialect()"),

    # Encryption (backend-specific)
    supports_encryption = function() FALSE,
    enable_encryption = function(conn, key) {
      stop("Encryption not supported by this backend")
    }
  )
)
```

2.4 SQLite Adapter

```r
# R/db_adapter_sqlite.R
```

```
SQLiteAdapter <- R6::R6Class("SQLiteAdapter",
  inherit = DatabaseAdapter,

  public = list(
    initialize = function(config) {
      super$initialize(config)
      if (!requireNamespace("RSQLite", quietly = TRUE)) {
        stop("RSQLite package required for SQLite backend")
      }
    },

    connect = function() {
      RSQLite::dbConnect(
        RSQLite::SQLite(),
        dbname = self$config$db_path
      )
    },

    pool = function() {
      pool::dbPool(
        RSQLite::SQLite(),
        dbname = self$config$db_path
      )
    },

    dialect = function() {
      list(
        auto_increment = "INTEGER PRIMARY KEY AUTOINCREMENT",
        timestamp_now = "datetime('now')",
        json_extract = function(col, path) {
          sprintf("json_extract(%s, '$.%s')", col, path)
        },
        boolean_true = "1",
        boolean_false = "0",
        concat = function(...) paste(..., sep = " || ")
      )
    },

    supports_encryption = function() {
      requireNamespace("RSQLCipher", quietly = TRUE)
    },

    enable_encryption = function(conn, key) {
      DBI::dbExecute(conn, sprintf("PRAGMA key = '%s'", key))
    }
  )
)
```

2.5 PostgreSQL Adapter

```
# R/db_adapter_postgres.R

PostgreSQLAdapter <- R6::R6Class("PostgreSQLAdapter",
  inherit = DatabaseAdapter,
```

```
  public = list(
    initialize = function(config) {
      super$initialize(config)
      if (!requireNamespace("RPostgres", quietly = TRUE)) {
        stop("RPostgres package required for PostgreSQL backend")
      }
    },

    connect = function() {
      RPostgres::dbConnect(
        RPostgres::Postgres(),
        host = self$config$db_host %||% "localhost",
        port = self$config$db_port %||% 5432,
        dbname = self$config$db_name,
        user = self$config$db_user,
        password = self$config$db_password
      )
    },

    pool = function() {
      pool::dbPool(
        RPostgres::Postgres(),
        host = self$config$db_host %||% "localhost",
        port = self$config$db_port %||% 5432,
        dbname = self$config$db_name,
        user = self$config$db_user,
        password = self$config$db_password,
        minSize = self$config$pool_min %||% 1,
        maxSize = self$config$pool_max %||% 10
      )
    },

    dialect = function() {
      list(
        auto_increment = "SERIAL PRIMARY KEY",
        timestamp_now = "CURRENT_TIMESTAMP",
        json_extract = function(col, path) {
          sprintf("%s->>'%s'", col, path)
        },
        boolean_true = "TRUE",
        boolean_false = "FALSE",
        concat = function(...) paste("CONCAT(", paste(..., collapse = ", "), ")")
      )
    },

    supports_encryption = function() {
      # PostgreSQL uses TLS for transport encryption
      # and pgcrypto for column-level encryption
      TRUE
    }
  )
)
```

2.6 DuckDB Adapter

```r
# R/db_adapter_duckdb.R

DuckDBAdapter <- R6::R6Class("DuckDBAdapter",
  inherit = DatabaseAdapter,

  public = list(
    initialize = function(config) {
      super$initialize(config)
      if (!requireNamespace("duckdb", quietly = TRUE)) {
        stop("duckdb package required for DuckDB backend")
      }
    },

    connect = function() {
      duckdb::dbConnect(
        duckdb::duckdb(),
        dbdir = self$config$db_path %||% ":memory:",
        read_only = self$config$read_only %||% FALSE
      )
    },

    pool = function() {
      # DuckDB handles concurrent reads efficiently
      # Return a connection wrapper compatible with pool interface
      conn <- self$connect()
      structure(
        list(
          conn = conn,
          fetch = function() conn,
          return = function(x) invisible(NULL)
        ),
        class = c("duckdb_pool", "list")
      )
    },

    dialect = function() {
      list(
        auto_increment = "INTEGER PRIMARY KEY",
        timestamp_now = "CURRENT_TIMESTAMP",
        json_extract = function(col, path) {
          sprintf("json_extract_string(%s, '$.%s')", col, path)
        },
        boolean_true = "TRUE",
        boolean_false = "FALSE",
        concat = function(...) paste(..., sep = " || ")
      )
    },

    supports_encryption = function() {
      # DuckDB supports encryption at database creation
      TRUE
    },
```

```
# DuckDB-specific: export to Parquet
export_parquet = function(conn, query_or_table, path) {
  if (grepl("\\s", query_or_table)) {
    # It's a query
    DBI::dbExecute(conn, sprintf(
      "COPY (%s) TO '%s' (FORMAT PARQUET)", query_or_table, path
    ))
  } else {
    # It's a table name
    DBI::dbExecute(conn, sprintf(
      "COPY %s TO '%s' (FORMAT PARQUET)", query_or_table, path
    ))
  }
},

# DuckDB-specific: export to CSV
export_csv = function(conn, query_or_table, path) {
  if (grepl("\\s", query_or_table)) {
    DBI::dbExecute(conn, sprintf(
      "COPY (%s) TO '%s' (HEADER, DELIMITER ',')", query_or_table, path
    ))
  } else {
    DBI::dbExecute(conn, sprintf(
      "COPY %s TO '%s' (HEADER, DELIMITER ',')", query_or_table, path
    ))
  }
},

# DuckDB-specific: import from Parquet
import_parquet = function(conn, path, table = NULL) {
  if (is.null(table)) {
    # Return as query result
    DBI::dbGetQuery(conn, sprintf(
      "SELECT * FROM read_parquet('%s')", path
    ))
  } else {
    # Create table
    DBI::dbExecute(conn, sprintf(
      "CREATE TABLE %s AS SELECT * FROM read_parquet('%s')",
      table, path
    ))
  }
},

# DuckDB-specific: import from CSV
import_csv = function(conn, path, table = NULL) {
  if (is.null(table)) {
    DBI::dbGetQuery(conn, sprintf(
      "SELECT * FROM read_csv_auto('%s')", path
    ))
  } else {
    DBI::dbExecute(conn, sprintf(
      "CREATE TABLE %s AS SELECT * FROM read_csv_auto('%s')",
```

```
          table, path
        ))
      }
    }
  )
)
```

Part 3: Configuration Schema

3.1 Config File Format

```yaml
# config.yml

# Database backend: sqlite, postgresql, or duckdb
db_backend: duckdb

# SQLite-specific (used when db_backend: sqlite)
sqlite:
  path: data/zzedc.db
  encryption_key: ${ZZEDC_DB_KEY}  # Optional, requires RSQLCipher

# DuckDB-specific (used when db_backend: duckdb)
duckdb:
  path: data/zzedc.duckdb

# PostgreSQL-specific (used when db_backend: postgresql)
postgresql:
  host: localhost
  port: 5432
  name: zzedc
  user: ${ZZEDC_DB_USER}
  password: ${ZZEDC_DB_PASSWORD}
  pool_min: 1
  pool_max: 10
  ssl_mode: require  # disable, allow, prefer, require, verify-ca, verify-full
```

3.2 Configuration Loading

```r
# R/config.R (updated)

load_config <- function(path = "config.yml") {

  config <- yaml::read_yaml(path)

  # Expand environment variables

config <- expand_env_vars(config)

  # Validate required fields based on backend
  validate_db_config(config)

  # Create adapter
  config$db_adapter <- create_db_adapter(config)
```

```
  structure(config, class = "zzedc_config")
}

expand_env_vars <- function(x) {
  if (is.character(x)) {
    # Replace ${VAR} with environment variable
    stringr::str_replace_all(x, "\\$\\{([^}]+)\\}", function(m) {
      var_name <- stringr::str_match(m, "\\$\\{([^}]+)\\}")[,2]
      Sys.getenv(var_name, unset = "")
    })
  } else if (is.list(x)) {
    lapply(x, expand_env_vars)
  } else {
    x
  }
}
```

Part 4: SQL Dialect Handling

4.1 Dialect-Aware Query Builder

```
# R/db_dialect.R

#' Build CREATE TABLE statement for current dialect
#' @param adapter Database adapter
#' @param table_name Table name
#' @param columns Named list of column definitions
build_create_table <- function(adapter, table_name, columns) {

  dialect <- adapter$dialect()

  col_defs <- mapply(function(name, def) {
    type <- switch(def$type,
      "id" = dialect$auto_increment,
      "text" = "TEXT",
      "varchar" = sprintf("VARCHAR(%d)", def$length %||% 255),
      "integer" = "INTEGER",
      "boolean" = "BOOLEAN",
      "timestamp" = "TIMESTAMP",
      "date" = "DATE",
      "json" = if (inherits(adapter, "PostgreSQLAdapter")) "JSONB" else "TEXT",
      def$type
    )

    constraints <- c()
    if (isTRUE(def$not_null)) constraints <- c(constraints, "NOT NULL")
    if (!is.null(def$default)) {
      default_val <- if (def$default == "now") {
        dialect$timestamp_now
      } else {
        sprintf("'%s'", def$default)
      }
```

```
      constraints <- c(constraints, sprintf("DEFAULT %s", default_val))
    }

    paste(c(name, type, constraints), collapse = " ")
  }, names(columns), columns, SIMPLIFY = FALSE, USE.NAMES = FALSE)

  sprintf(
    "CREATE TABLE IF NOT EXISTS %s (\n  %s\n)",
    table_name,
    paste(unlist(col_defs), collapse = ",\n  ")
  )
}

#' Generate parameterized INSERT statement
#' @param adapter Database adapter
#' @param table_name Table name
#' @param columns Column names
build_insert <- function(adapter, table_name, columns) {

  placeholders <- if (inherits(adapter, "PostgreSQLAdapter")) {
    paste0("$", seq_along(columns))
  } else {
    rep("?", length(columns))
  }

  sprintf(
    "INSERT INTO %s (%s) VALUES (%s)",
    table_name,
    paste(columns, collapse = ", "),
    paste(placeholders, collapse = ", ")
  )
}
```

4.2 Common Query Patterns

```
# R/db_queries.R

#' Get current timestamp expression for dialect
db_now <- function(adapter) {
  adapter$dialect()$timestamp_now
}

#' Build JSON extract expression
db_json_get <- function(adapter, column, path) {
  adapter$dialect()$json_extract(column, path)
}

#' Build boolean literal
db_bool <- function(adapter, value) {
  if (value) adapter$dialect()$boolean_true
  else adapter$dialect()$boolean_false
}
```

Part 5: Migration Strategy

5.1 Schema Migration System

```r
# R/db_migration.R

#' Run database migrations
#' @param adapter Database adapter
#' @param migrations_dir Directory containing migration files
run_migrations <- function(adapter, migrations_dir = "migrations") {

  conn <- adapter$connect()
  on.exit(adapter$disconnect(conn))

  # Ensure migrations table exists
  ensure_migrations_table(adapter, conn)

  # Get applied migrations
  applied <- DBI::dbGetQuery(conn,
    "SELECT migration_id FROM schema_migrations ORDER BY migration_id"
  )$migration_id

  # Get pending migrations
  files <- list.files(migrations_dir, pattern = "^\\d+.*\\.sql$",
                      full.names = TRUE)
  files <- sort(files)

  for (file in files) {
    migration_id <- basename(file)
    if (migration_id %in% applied) next

    message("Applying migration: ", migration_id)

    sql <- readLines(file, warn = FALSE)
    sql <- paste(sql, collapse = "\n")

    # Handle dialect-specific sections
    sql <- process_dialect_blocks(sql, adapter)

    adapter$transaction(conn, {
      # Split on semicolons and execute each statement
      statements <- split_sql_statements(sql)
      for (stmt in statements) {
        if (nchar(trimws(stmt)) > 0) {
          DBI::dbExecute(conn, stmt)
        }
      }

      # Record migration
      DBI::dbExecute(conn,
        "INSERT INTO schema_migrations (migration_id, applied_at)
         VALUES (?, CURRENT_TIMESTAMP)",
        params = list(migration_id)
      )
    })
```

```
  }
}

#' Process dialect-specific blocks in SQL
#' @param sql SQL content
#' @param adapter Database adapter
process_dialect_blocks <- function(sql, adapter) {

  backend <- class(adapter)[1]
  backend_tag <- switch(backend,
    "SQLiteAdapter" = "sqlite",
    "PostgreSQLAdapter" = "postgresql",
    "DuckDBAdapter" = "duckdb"
  )

  # Pattern: -- @sqlite: ... -- @end or -- @postgresql: ... -- @end
  # Keep matching blocks, remove non-matching

  # Remove blocks for other backends
  other_backends <- setdiff(c("sqlite", "postgresql", "duckdb"), backend_tag)
  for (other in other_backends) {
    pattern <- sprintf("-- @%s:.*?-- @end", other)
    sql <- gsub(pattern, "", sql, perl = TRUE)
  }

  # Unwrap blocks for current backend
  pattern <- sprintf("-- @%s:(.*?)-- @end", backend_tag)
  sql <- gsub(pattern, "\\1", sql, perl = TRUE)

  sql
}
```

5.2 Example Migration File

```
-- migrations/001_initial_schema.sql

-- Users table
CREATE TABLE IF NOT EXISTS users (
-- @sqlite:
  user_id INTEGER PRIMARY KEY AUTOINCREMENT,
-- @end
-- @postgresql:
  user_id SERIAL PRIMARY KEY,
-- @end
  username VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(255),
  email VARCHAR(255),
  role VARCHAR(50) NOT NULL DEFAULT 'user',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Audit trail
CREATE TABLE IF NOT EXISTS audit_trail (
-- @sqlite:
  audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
-- @end
-- @postgresql:
  audit_id SERIAL PRIMARY KEY,
-- @end
  event_type VARCHAR(100) NOT NULL,
  entity_type VARCHAR(100),
  entity_id INTEGER,
  user_id INTEGER REFERENCES users(user_id),
  old_value TEXT,
  new_value TEXT,
  ip_address VARCHAR(45),
  previous_hash VARCHAR(64),
  current_hash VARCHAR(64) NOT NULL,
-- @sqlite:
  created_at TEXT DEFAULT (datetime('now')),
-- @end
-- @postgresql:
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
-- @end
);

CREATE INDEX idx_audit_entity ON audit_trail(entity_type, entity_id);
CREATE INDEX idx_audit_user ON audit_trail(user_id);
CREATE INDEX idx_audit_created ON audit_trail(created_at);
```

Part 6: Refactoring Existing Code

6.1 Connection Management Updates

Before (current zzedc):

```r
connect_db <- function(db_path) {
  RSQLite::dbConnect(RSQLite::SQLite(), db_path)
}
```

After (refactored):

```r
connect_db <- function(config = NULL) {
  config <- config %||% get_global_config()
  config$db_adapter$connect()
}

get_db_pool <- function(config = NULL) {
  config <- config %||% get_global_config()
  config$db_adapter$pool()
}
```

6.2 Query Pattern Updates

Before:

```r
log_audit_event <- function(conn, event_type, entity_type, entity_id,
                            user_id, old_value, new_value) {
  DBI::dbExecute(conn,
    "INSERT INTO audit_trail
     (event_type, entity_type, entity_id, user_id, old_value, new_value,
      created_at)
     VALUES (?, ?, ?, ?, ?, ?, datetime('now'))",
    params = list(event_type, entity_type, entity_id, user_id,
                  old_value, new_value)
  )
}
```

After:

```r
log_audit_event <- function(conn, event_type, entity_type, entity_id,
                            user_id, old_value, new_value,
                            config = NULL) {
  config <- config %||% get_global_config()
  adapter <- config$db_adapter

  sql <- sprintf(
    "INSERT INTO audit_trail
     (event_type, entity_type, entity_id, user_id, old_value, new_value,
      created_at)
     VALUES (?, ?, ?, ?, ?, ?, %s)",
    db_now(adapter)
  )

  adapter$execute(conn, sql,
    params = list(event_type, entity_type, entity_id, user_id,
                  old_value, new_value)
  )
}
```

6.3 Files to Refactor

File	Changes Required	Effort
db_connection.R	Replace RSQLite with adapter	1 day
db_migration.R	Add dialect handling	2 days
audit_logging.R	Use db_now()	0.5 day
audit_enhanced.R	Use db_now()	0.5 day
validation_rules.R	Use adapter queries	0.5 day
consent.R	Use adapter queries	0.5 day
dsar.R	Use adapter queries	0.5 day
erasure.R	Use adapter queries	0.5 day
electronic_signatures.R	Use adapter queries	0.5 day
All other R/*.R	Minor query updates	2 days
gsheets_dd_builder.R	Dialect-aware DDL	1 day
Tests	Update for multi-backend	2 days

Total refactoring effort: ~12 days

Part 7: Testing Strategy

7.1 Multi-Backend Test Infrastructure

```r
# tests/testthat/helper-db.R

# Get test backends from environment or default to SQLite
get_test_backends <- function() {
  backends <- Sys.getenv("ZZEDC_TEST_BACKENDS", "sqlite")
  strsplit(backends, ",")[[1]]
}

# Create test config for each backend
test_config <- function(backend) {
  switch(backend,
    sqlite = list(
      db_backend = "sqlite",
      db_path = tempfile(fileext = ".db")
    ),
    postgresql = list(
      db_backend = "postgresql",
      db_host = Sys.getenv("ZZEDC_TEST_PG_HOST", "localhost"),
      db_port = as.integer(Sys.getenv("ZZEDC_TEST_PG_PORT", "5432")),
      db_name = Sys.getenv("ZZEDC_TEST_PG_DB", "zzedc_test"),
      db_user = Sys.getenv("ZZEDC_TEST_PG_USER", "postgres"),
      db_password = Sys.getenv("ZZEDC_TEST_PG_PASSWORD", "")
    ),
    duckdb = list(
      db_backend = "duckdb",
      db_path = tempfile(fileext = ".duckdb")
    )
  )
}

# Run test across all configured backends
with_test_backends <- function(code) {
  for (backend in get_test_backends()) {
    test_that(sprintf("[%s] %s", backend, deparse(substitute(code))), {
      config <- test_config(backend)
      config$db_adapter <- create_db_adapter(config)
      withr::local_options(list(zzedc.config = config))
      eval(code)
    })
  }
}
```

7.2 Example Multi-Backend Test

```r
# tests/testthat/test-audit.R

with_test_backends({
  config <- getOption("zzedc.config")
  conn <- config$db_adapter$connect()
  on.exit(config$db_adapter$disconnect(conn))
```

```
  # Setup
  run_migrations(config$db_adapter)

  # Test
  log_audit_event(conn, "TEST", "test_entity", 1, 1, NULL, "new",
                  config = config)

  result <- config$db_adapter$query(conn,
    "SELECT * FROM audit_trail WHERE event_type = 'TEST'")

  expect_equal(nrow(result), 1)
  expect_equal(result$entity_type, "test_entity")
})
```

7.3 CI Configuration

```
# .github/workflows/test.yml

jobs:
  test:
    strategy:
      matrix:
        backend: [sqlite, postgresql]

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_DB: zzedc_test
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v4

      - uses: r-lib/actions/setup-r@v2

      - name: Install dependencies
        run: |
          install.packages(c("RSQLite", "RPostgres", "duckdb", "pool"))

      - name: Run tests
        env:
          ZZEDC_TEST_BACKENDS: ${{ matrix.backend }}
          ZZEDC_TEST_PG_PASSWORD: postgres
        run: |
          testthat::test_local()
```

Part 8: DuckDB-Specific Features

When DuckDB is the primary backend, these additional capabilities are available.

8.1 Data Export Functions

```r
# R/db_export.R

#' Export data to Parquet format (DuckDB only)
#' @param conn Database connection
#' @param query_or_table SQL query or table name
#' @param path Output file path
#' @param config zzedc config
export_parquet <- function(conn, query_or_table, path, config = NULL) {
  config <- config %||% get_global_config()

  if (!inherits(config$db_adapter, "DuckDBAdapter")) {
    stop("Parquet export requires DuckDB backend")
  }

  config$db_adapter$export_parquet(conn, query_or_table, path)
}

#' Export adverse events for statistical analysis
#' @param conn Database connection
#' @param output_dir Output directory
export_analysis_datasets <- function(conn, output_dir, config = NULL) {
  config <- config %||% get_global_config()
  adapter <- config$db_adapter

  if (!inherits(adapter, "DuckDBAdapter")) {
    # Fall back to CSV for non-DuckDB backends
    export_analysis_datasets_csv(conn, output_dir, config)
    return(invisible(NULL))
  }

  # DuckDB: use native Parquet export
  dir.create(output_dir, showWarnings = FALSE, recursive = TRUE)

  adapter$export_parquet(conn, "subjects", file.path(output_dir, "subjects.parquet"))
  adapter$export_parquet(conn, "adverse_events", file.path(output_dir, "ae.parquet"))
  adapter$export_parquet(conn, "consent", file.path(output_dir, "consent.parquet"))

  # Export with query for derived datasets
  adapter$export_parquet(conn,
    "SELECT * FROM subjects WHERE status = 'completed'",
    file.path(output_dir, "completers.parquet")
  )
}
```

8.2 Data Import Functions

```r
#' Import Parquet file directly into database (DuckDB only)
#' @param conn Database connection
#' @param path Parquet file path
```

```r
#' @param table Target table name (NULL to return data frame)
import_parquet <- function(conn, path, table = NULL, config = NULL) {
  config <- config %||% get_global_config()

  if (!inherits(config$db_adapter, "DuckDBAdapter")) {
    # Fall back to arrow for non-DuckDB
    df <- arrow::read_parquet(path)
    if (!is.null(table)) {
      DBI::dbWriteTable(conn, table, df)
    }
    return(df)
  }

  config$db_adapter$import_parquet(conn, path, table)
}
```

8.3 Performance Advantages

DuckDB excels at analytical queries common in clinical trials:

```r
# Enrollment summary by site - fast with DuckDB columnar storage
enrollment <- db_query(pool, "
  SELECT
    site_id,
    COUNT(*) as enrolled,
    COUNT(*) FILTER (WHERE status = 'completed') as completed,
    COUNT(*) FILTER (WHERE status = 'withdrawn') as withdrawn
  FROM subjects
  GROUP BY site_id
  ORDER BY enrolled DESC
")

# Adverse event frequency - benefits from columnar scan
ae_summary <- db_query(pool, "
  SELECT
    ae_term,
    COUNT(*) as count,
    COUNT(DISTINCT subject_id) as subjects_affected
  FROM adverse_events
  GROUP BY ae_term
  ORDER BY count DESC
  LIMIT 20
")
```

8.4 R Integration

DuckDB integrates seamlessly with R data analysis workflows:

```r
# Direct query to data frame (zero-copy when possible)
library(duckdb)

conn <- config$db_adapter$connect()

# Query result directly usable in R
subjects <- DBI::dbGetQuery(conn, "SELECT * FROM subjects")
```

```r
# Or use dplyr interface
library(dplyr)
subjects_tbl <- tbl(conn, "subjects")
completers <- subjects_tbl |>
  filter(status == "completed") |>
  collect()
```

Part 9: Migration from bbl

9.1 What Transfers from bbl to zzedc

bbl Feature	zzedc Location	Notes
PostgreSQL connection	db_adapter_postgres.R	Generalized
Connection pooling	db_adapter.R pool()	All backends
Transaction logging	Audit trail	Already in zzedc
Hash-chain audit	audit_logging.R	Compare implementations

9.2 bbl Code to Review

Some bbl implementations may be more complete:

```r
# Compare these files:
# bbl/R/audit.R vs zzedc/R/audit_logging.R
# bbl/R/signatures.R vs zzedc/R/electronic_signatures.R
# bbl/R/versioning.R vs zzedc/R/version_control.R
```

9.3 Deprecation Timeline

1. **Phase 1**: Add database abstraction to zzedc (this plan)
2. **Phase 2**: Verify feature parity with bbl
3. **Phase 3**: Mark bbl as deprecated
4. **Phase 4**: Archive bbl repository

Part 10: Implementation Schedule

Week 1: Core Abstraction

- ☐ Create db_adapter.R base class
- ☐ Create db_adapter_sqlite.R
- ☐ Create db_adapter_postgres.R
- ☐ Update config.R for multi-backend

Week 2: Dialect & Migration

- ☐ Create db_dialect.R
- ☐ Update db_migration.R for dialect blocks
- ☐ Convert existing migrations to multi-dialect
- ☐ Test migration system

Week 3: Code Refactoring

- ☐ Update db_connection.R
- ☐ Update all R/*.R files with adapter pattern
- ☐ Update gsheets_dd_builder.R for PostgreSQL DDL
- ☐ Update Shiny app connection handling

Week 4: Testing & DuckDB

- ☐ Create multi-backend test infrastructure
- ☐ Update existing tests
- ☐ Add PostgreSQL CI pipeline
- ☐ Implement DuckDB adapter
- ☐ Add DuckDB-specific export functions

Week 5: Documentation & Polish

- ☐ Update user documentation
- ☐ Write migration guide from bbl
- ☐ Performance testing
- ☐ Final review and cleanup

Part 11: Effort Summary

Task	Days
Core abstraction layer	3
SQL dialect handling	2
Migration system updates	2
Code refactoring	5
Test infrastructure	2
DuckDB adapter + export features	2
Documentation	2
Total	**18 days (~4 weeks)**

Comparison:

Approach	Effort
Expand bbl separately	33 weeks
Refactor zzedc for multi-backend	4 weeks

Savings: 29 weeks

Document History

Version	Date	Changes
1.0	Jan 2026	Initial plan

Version	Date	Changes
1.1	Jan 2026	DuckDB as full backend, deployment-focused selection guide